TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN · BASES DE DATOS II

TRABAJO PRÁCTICO · UNIDAD 7: NoSQL Y MONGODB

Proyecto: Food Store | **Alumno:** Máximo Juan Cruz Quiroga | **Docente:** Sergio Neira | UTN FRM 2026

Parte 1 · Modelo Documental y Migración

Consigna 1: Identificación Relacional y Diseño Documental

En PostgreSQL 17, el circuito de órdenes está normalizado en 4 tablas: pedidos (PK id_pedido, FK id_cliente, forma_pago, fecha_pedido, estado), detalles_pedido (PK compuesta id_pedido + id_producto, cantidad, precio_unitario), clientes (id_cliente, nombre, apellido, correo_electronico) y productos (id_producto, nombre, precio_actual). Para MongoDB se adoptó el **patrón de documentos embebidos (Embedded Document Pattern)** en la colección pedidos. Cada documento concentra la orden completa, integrando un objeto embebido cliente y un arreglo lineas con los renglones comprados, preservando los identificadores de origen (id_pedido, id_cliente, id_producto) como números enteros para trazabilidad bidireccional.


```
Colección: foodstore_nosql.pedidos { "id_pedido": 1, "fecha_pedido": ISODate("2026-10-15T12:30:00Z"), "forma_pago": "TARJETA", "estado": "ENTREGADO", "cliente": { "id_cliente": 1, "nombre": "Juan Pérez", "correo_electronico": "juan.perez@email.com" }, "lineas": [ { "id_producto": 1, "nombre": "Hamburguesa Clásica", "cantidad": 2, "precio_unitario": 4500.00, "subtotal": 9000.00 }, { "id_producto": 3, "nombre": "Papas Fritas Medianas", "cantidad": 1, "precio_unitario": 2200.00, "subtotal": 2200.00 }, { "id_producto": 4, "nombre": "Gaseosa Cola 500ml", "cantidad": 2, "precio_unitario": 1500.00, "subtotal": 3000.00 } ], "total": 14200.00 }
```


Consigna 2: Base de Datos y Conjunto de Datos Migrados

Se creó la base foodstore_nosql y la colección pedidos. La migración se ejecutó mediante insertMany() en migracion_pedidos.js, migrando 5 pedidos con líneas y datos esenciales del cliente:

ID	Fecha	Cliente	Estado	Forma Pago	Ítems del Pedido	Total ($)
1	15/10/26	Juan Pérez	ENTREGADO	TARJETA	2x Hamb. Clásica, 1x Papas Med., 2x Gaseosa	**14.200,00**
2	16/10/26	María Gómez	PENDIENTE	EFECTIVO	1x Hamb. Clásica, 1x Gaseosa Cola	**6.000,00**
3	16/10/26	Carlos López	ENTREGADO	TRANSFER.	2x Hamb. Doble Queso, 2x Cerveza Art.	**18.000,00**
4	17/10/26	Juan Pérez	CANCELADO	TARJETA	1x Hamb. Clásica, 1x Hamb. Doble Queso	**10.700,00**
5	18/10/26	María Gómez	ENTREGADO	TRANSFER.	2x Papas Med., 2x Gaseosa Cola	**7.400,00**

Consigna 3: Reglas de Integridad y Justificación de Embebido

1. Congelamiento Histórico de Precios y Cantidades (Reglas R4 y R5): En Food Store, el precio unitario y subtotal deben quedar inalterables respecto al momento de compra, independientemente de incrementos futuros en la tabla productos.
2. Integridad de Dominio y No Negatividad (R5): Cantidades estrictamente positivas ($cantidad \ge 1$), precios y subtotales no negativos ($precio \ge 0$), blindados con $jsonSchema.
3. Experimento de Validación Cruzada ($expr): Mediante prueba_validacion_expr.js se verificó en laboratorio que MongoDB puede forzar reglas entre campos (total = SUM(subtotales) y subtotal = cant * precio) rechazando inconsistencias con Código 121 (*Document failed validation*). En PostgreSQL, esta coherencia es nativa vía columnas generadas.
Justificación de Embebido: (a) *Lectura Atómica en 1 operación (Single-Document Read):* el 95% de las consultas piden el pedido completo; embeber elimina la penalización de JOINs ($lookup). (b) *Atomicidad nativa:* las escrituras en un solo documento son atómicas por definición en MongoDB. (c) *Snapshot de cliente:* los datos del comprador no se alteran si luego modifica su perfil.

Parte 2 · Consultas y Verificación

Consigna 4: Ejecución en MongoDB (consultas_mongodb.js)

```
// 4.a Mostrar todos los pedidos (id y estado) db.pedidos.find({}, { id_pedido: 1, estado: 1, fecha_pedido: 1, total: 1, _id: 0 }).sort({ id_pedido: 1 }); // Resultado: 5 documentos recuperados (IDs: 1, 2, 3, 4, 5). // 4.b Filtrar por estado existente ('ENTREGADO') db.pedidos.find({ estado: "ENTREGADO" }, { id_pedido: 1, estado: 1, "cliente.nombre": 1, total: 1, _id: 0 }); // Resultado: 3 documentos recuperados (IDs: 1, 3, 5). // 4.c Buscar pedidos que contengan el producto 1 ('Hamburguesa Clásica') dentro del arreglo db.pedidos.find({ "lineas.id_producto": 1 }, { id_pedido: 1, "lineas.nombre": 1, total: 1, _id: 0 }); // Resultado: 3 documentos recuperados con Multikey Index (IDs: 1, 2, 4). // 4.d Buscar pedidos cuyo total sea superior a $10.000,00 ARS db.pedidos.find({ total: { $gt: 10000.00 } }, { id_pedido: 1, total: 1, "cliente.nombre": 1, _id: 0 }); // Resultado: 3 documentos recuperados (ID 3: $18.000, ID 1: $14.200, ID 4: $10.700). // Justificación de umbral: $10.000 ARS aísla compras de alto volumen de consumos individuales básicos.
```

Consigna 5: Consultas Equivalentes en PostgreSQL y Tabla de Verificación

En PostgreSQL (consultas_postgresql.sql), resolver estas consultas exige JOINs entre pedidos, clientes, detalles_pedido y productos:

```
-- Equivalente 5.b: Pedidos con estado = 'ENTREGADO' SELECT p.id_pedido, p.estado, c.nombre || ' ' || c.apellido AS cliente, SUM(dp.cantidad * dp.precio_unitario) AS total FROM pedidos p JOIN clientes c ON c.id_cliente = p.id_cliente JOIN detalles_pedido dp ON dp.id_pedido = p.id_pedido WHERE p.estado = 'ENTREGADO' GROUP BY p.id_pedido, p.estado, c.nombre, c.apellido ORDER BY p.id_pedido; -- Equivalente 5.c: Pedidos que contienen id_producto = 1 (Hamburguesa Clásica) SELECT DISTINCT p.id_pedido, p.estado, c.nombre || ' ' || c.apellido AS cliente FROM pedidos p JOIN clientes c ON c.id_cliente = p.id_cliente JOIN detalles_pedido dp ON dp.id_pedido = p.id_pedido WHERE dp.id_producto = 1 ORDER BY p.id_pedido;
```

Consulta Evaluada	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?	IDs Recuperados
Estado seleccionado ('ENTREGADO')	**3**	**3**	**SÍ**	[1, 3, 5]
Producto seleccionado (id_producto = 1)	**3**	**3**	**SÍ**	[1, 2, 4]

Explicación de las diferencias mecánicas: En PostgreSQL, la consulta (b) requiere agrupar filas y calcular agregaciones en caliente con SUM(), mientras que la consulta (c) exige cruzar 4 tablas con JOIN y aplicar DISTINCT para desduplicar pedidos con múltiples renglones. En MongoDB, la consulta (b) lee directamente el atributo precalculado estado, y la consulta (c) aprovecha el **Multikey Index** sobre lineas.id_producto, resolviendo la búsqueda sin combinaciones ni desduplicación de resultados.

Parte 3 · Evaluación de la Tecnología e Inteligencia Artificial

Consigna 6: Consulta Realizada a la Inteligencia Artificial

Se consultó a un asistente de IA de arquitectura especializada bajo el siguiente requerimiento explícito:

Prompt enviado a la IA:
"Actuá como Arquitecto de Bases de Datos senior. En nuestro sistema Food Store (PostgreSQL 17), los pedidos se modelan en tablas relacionales normalizadas (pedidos, detalles_pedido, clientes). El 95% de las consultas leen pedidos completos para emisión de comprobantes y detalle de compra. Además, una regla crítica del negocio es congelar el precio unitario y cantidad al momento de la compra para que futuros cambios en el catálogo no alteren el historial. Evaluá técnica y comparativamente si conviene mantener PostgreSQL o migrar este subconjunto a una colección documental en MongoDB usando un modelo embebido. Contrastá rendimiento de lectura, consistencia transaccional y complejidad operativa."

Resumen de la Respuesta Recibida de la IA:

La IA argumentó enfáticamente a favor de MongoDB, señalando que *"el modelo relacional normalizado es ineficiente para pedidos debido a la necesidad de costosos JOINs entre pedidos, detalles y productos"*. Propuso migrar la totalidad del e-commerce a colecciones de MongoDB y formuló dos afirmaciones técnicas categóricas: (1) que *"MongoDB no soporta transacciones ACID ni integridad referencial, por lo que las garantías de consistencia financiera deben delegarse exclusivamente a código en Node.js o Python"*; y (2) que *"migrar a MongoDB reducirá a la mitad la latencia de lectura en cualquier escenario debido a que prescinde de esquemas rígidos"*.

Consigna 7: Contraste Crítico de Afirmaciones y Conclusión Técnica

Contraste 1: Sobre la supuesta ausencia de transacciones ACID en MongoDB.
Refutación con Documentación Oficial de MongoDB: La afirmación de la IA es anacrónica y técnicamente falsa. En primer lugar, la documentación oficial de MongoDB (*Data Modeling: Atomicity of Operations*) estipula que toda modificación sobre un único documento es **estrictamente atómica e íntegra**. En nuestro modelo de pedidos embebidos, la inserción del pedido con todas sus líneas es atómica por naturaleza, sin requerir transacciones complejas. En segundo lugar, desde MongoDB 4.0 (y perfeccionado en 4.2+ para sharding), el motor soporta **Transacciones Multi-Documento ACID** completas con aislamiento snapshot y rollback automático ante errores de red o caídas del servidor.

Contraste 2: Sobre la afirmación de que MongoDB siempre reducirá la latencia a la mitad.
Refutación con Evidencia de Laboratorio y Documentación PostgreSQL: Esta afirmación constituye una generalización errónea. En nuestro entorno de laboratorio con PostgreSQL 17, al consultar un pedido por su clave primaria indexada junto a un índice en detalles_pedido(id_pedido), el planificador ejecuta un Index Scan y resuelve la consulta en **menos de 1 milisegundo** mediante páginas cacheadas en el *Buffer Cache* de memoria RAM. Asimismo, PostgreSQL soporta de forma nativa el tipo JSONB con funciones de agregación (json_build_object / json_agg), lo que permite emitir exactamente el mismo documento jerárquico que MongoDB sin penalizaciones perceptibles de latencia.

Conclusión Técnica Final (189 palabras):

Para el sistema Food Store, la decisión arquitectónica óptima es adoptar **Persistencia Políglota (Polyglot Persistence)**, combinando ambos motores según el patrón de acceso:

1. **PostgreSQL como Almacén de Registro Transaccional (System of Record):** Debe conservarse para el catálogo maestro de productos, categorías, gestión de clientes, control de inventario y facturación fiscal. Estas áreas exigen restricciones de clave foránea estrictas (ON DELETE RESTRICT), auditoría nativa, roles de seguridad granulares y consistencia ACID inmediata ante escrituras concurrentes.

2. **MongoDB como Repositorio Documental de Órdenes (Order History & Read Store):** El subconjunto de pedidos cerrados y despachados se beneficia enormemente del modelo embebido en MongoDB. Almacenar el pedido desnormalizado con sus líneas y el snapshot del cliente permite que la aplicación web móvil y el servicio de atención consulten órdenes completas en un único ciclo de lectura (find), sin costo de JOINs y con escalabilidad horizontal natural.

Migrar la totalidad de Food Store a NoSQL introduciría complejidad innecesaria al replicar la integridad referencial en la capa de aplicación; en cambio, una arquitectura híbrida capitaliza lo mejor de ambos paradigmas.